MLOps compact cheatsheet

STEP 1 Set up and plan

1.1 Set up your computer for machine learning

The toolbox Python 3.12 or newer, terminal, .venv per project, pip or uv, Git, VS Code, later Docker and a GPU or Colab.

PATH The first match in PATH runs. activate puts .venv first. python -m pip keeps pip and python together.

Per system macOS and Linux: python3, .venv/bin. Windows: python or py, .venv\Scripts, execution policy. ChromeOS: Linux container, Debian steps.

Reproducible pip freeze > requirements.txt, pip install -r requirements.txt. Commit requirements, ignore .venv. Never sudo pip.

Docker Desktop on macOS and Windows (WSL 2), Engine on Linux, docker group, docker run hello-world.

GPU NVIDIA: driver, nvidia-smi, CUDA build of PyTorch. Apple silicon: MPS. Chromebook: Colab or Kaggle.

EXAM Two names, one idea: macOS, Linux and ChromeOS call it `python3`, Windows calls it `python` or `py`. Inside an activated virtual environment, `python` works everywhere.

EXAM The program shows what python -m venv creates: a bin folder (Scripts on Windows), include and lib folders, and pyvenv.cfg. The new python reports that it is inside a virtual environment, its packages go to the environment's own site-packages, and include-system-site-packages = false means it cannot see packages installed for the whole computer.

1.2 The MLOps lifecycle and reproducible projects

MLOps Version, test, deploy, monitor and retrain ML. DevOps plus data and models.

Levels 0 manual, 1 automated training pipeline (continuous training), 2 CI/CD for the pipeline.

Reproducible Fix seeds, report spread over seeds, save run records: data fingerprint, params, metrics, versions.

Versions Code in Git, data by hash (DVC), models in a registry, environment in requirements or images. Lineage.

Tests Data, features, model quality and behaviour, infrastructure. Notebooks to tested functions.

Why it matters Models decay as data changes. Hidden technical debt lives outside the model code.

An ML system
behaviour = code + data + model + configuration

Continuous training
new data → retrain → evaluate → register → deploy

EXAM The first block changes every time you run it, which is the point. With the seed fixed at 42, five runs give exactly the same accuracy, 0.972, five times. Over ten different seeds the accuracy has mean 0.956 and standard deviation 0.014, ranging from 0.937 to 0.972, so a difference of 0.01 between two models trained once means nothing.

EXAM The original breast cancer data gives a 5 fold accuracy of 0.981. Flipping 40 labels (7 percent) drops it to 0.882 with the same number of rows. Losing about 30 percent of the rows (399 left) gives 0.975. Each version has a different fingerprint, so the run record shows exactly which one a model saw.

STEP 2 Build and ship

2.1 Pipelines and saving models

Pipeline Preprocessing plus model in one object. ColumnTransformer: numbers impute and scale, categories one hot.

fit and transform Steps learn on training rows only, reuse at predict. handle_unknown='ignore' for unseen categories.

Leakage No step may see test rows. Selection outside CV: 0.85 on noise, inside: 0.48. Split by time and group too.

Saving joblib.dump the whole pipeline plus JSON metadata (features, classes, versions). Load trusted files only.

Formats joblib for own Python services, skops for safe sharing, ONNX for other languages.

Input contract Check presence, type, range, unknown categories, extra fields. Reject with 422.

Standard scaling $z = (x - \mu_{train}) / \sigma_{train}$

Median imputation
missing x → median of training x

One hot columns
one column per category seen in training

EXAM The churn pipeline reaches a 5 fold ROC AUC of 0.751 and a test accuracy of 0.69. After preprocessing, 5 raw columns have become 9 (3 scaled numbers and 6 one hot columns, with names such as cat__plan_basic). A new customer with a missing bill and a plan called student, which the model never saw, still gets a churn probability of 0.782 instead of an error.

EXAM On 100 rows of random noise with random labels, choosing the 20 best of 5000 columns before cross validation reports an accuracy of 0.85. Doing the same selection inside the pipeline reports 0.48, the honest answer for labels that are pure chance.

EXAM The fitted iris pipeline takes about 1.5 KB on disk. After loading, its predictions match the original on every row, and the flower [5.9, 3.0, 5.1, 1.8] is predicted as virginica. The JSON file records scikit-learn 1.8.0, which the serving side should match.

2.2 Serving models with Flask, FastAPI and Django

Model API Load once. /health plus POST /predict. JSON in and out. 200 success, 422 bad content, 405 wrong method, 500 is a bug.

Frameworks Flask small. FastAPI checks Pydantic classes, docs at /docs. Django for a model inside a full web app.

Test then call Test client in one process. Then curl (macOS, Linux, ChromeOS) or Invoke-RestMethod and curl.exe (Windows).

Online or batch Online answers during the visit. Batch scores on a schedule, cheaper, less fresh. Read big files in chunks.

Production servers Gunicorn on macOS and Linux, Waitress on Windows, Uvicorn for FastAPI everywhere. Several workers. Never app.run in production.

Capacity L = λW. Workers = λ × latency / target use. M/M/1: W = 1/(μ − λ) explodes near capacity.

Latency
time from request sent to response received

Throughput of one worker
requests per second = 1000 / latency in ms

Little's law $L = \lambda \times W$

Workers needed
workers = λ × latency / target use

EXAM One bad request produced three precise errors at once: sepal_length must be greater than 0, sepal_width is not a number, and petal_width is missing. The batch route scored two flowers in a single predict_proba call, virginica at 0.703 and setosa at 0.981.

TRAP Django gives the same answers as Flask (200 with virginica at 0.703, 422 for bad JSON), and @require_POST turns a GET into 405 without running the view. In a real project the view lives in an app inside `django-admin startproject`, and the settings come from environment variables, never a SECRET_KEY written in the code.

EXAM A queue does not grow in a straight line. One worker that can serve 50 requests per second answers in 25 ms at 10 per second, 100 ms at 40 per second, and 1 full second at 49 per second. That is why services keep spare capacity.

2.3 Docker and CI/CD for models

Image and container Image = OS layer, Python, packages, code, model. Container = running copy. Registry stores images. -p host:container.

Dockerfile order Base, requirements, pip install, then code. Code change: 2.2 s instead of 32 s. Non root USER, HEALTHCHECK, CMD.

Per system Docker Desktop on macOS and Windows (WSL 2), Engine plus docker group on Linux, Engine in the Linux environment on ChromeOS. Apple silicon: --platform linux/amd64.

Model tests Quality gate, known examples, invariance, directional, latency. A constant model passes invariance, so use several kinds.

CI/CD GitHub Actions: install, train, pytest, build, smoke test, push on main. First failure stops. Tag images by commit, secrets never in files.

Numbers Flaky suite $(1 - p)^n$. Accuracy SE $\sqrt{p(1 - p)/n}$. Canary $n = 4p(1 - p)/d^2$.

Chance a suite of flaky tests is green
$P(\text{green}) = (1 - p)^n$

Uncertainty of a test accuracy
$SE = \sqrt{(p(1 - p) / n)}$

EXAM Real timings of this project: with the good order, a one line change in app.py rebuilt in 2.2 seconds in total, because pip install came from the cache. With COPY . . before pip install, the same change took 32 seconds, because the 18 second install ran again.

TRAP The broken model scored 0.2 and failed three of the four checks, so CI blocks it. Look closely: it still passed the invariance test, because a model that ignores its input is perfectly stable. One kind of test is never enough.

STEP 3 Operate and improve

3.1 Experiment tracking and the model registry

Tracking Every run: parameters, metrics, artifacts, data fingerprint, Git commit, environment. Compare only runs on the same data.

MLflow start_run, log_params, log_metrics, sklearn.log_model, search_runs. mlflow ui --backend-store-uri sqlite:///mlflow.db, port 5001 on macOS.

Selection bias Best of many on one validation set flatters itself (0.615 vs 0.530). Untouched test set, report once. Gap about $\sigma\sqrt{(2 \ln n)}$.

Registry Frozen numbered versions with lineage, movable aliases. Services load models:/name@champion.

Promotion Same held out data, margin gate above noise, behaviour and fairness tests, human approval, logged reason.

Rollback Move the alias back. No retraining, no rebuild.

Random search coverage
$P(\text{at least one of n runs in the top q}) = 1 - (1 - q)^n$

Expected best of n noisy scores
about $\sigma\sqrt{(2 \ln n)}$ above the mean

EXAM Four runs were logged and sorted by validation AUC. C = 1.0 won with 0.9952, was registered as version 1 of breast-cancer-classifier with the alias champion, and loading models:/breast-cancer-classifier@champion gave back exactly the same AUC.

TRAP The labels are coin flips, so every model is truly 50 percent accurate. The best of 1000 runs still showed 0.615 on validation, then 0.530 on fresh test data. With 200 validation rows one standard error is 0.035, and the best of 1000 lands about three standard errors above the truth.

EXAM v2 (AUC 0.8844) was kept but not promoted. v3 (0.9942) beat v1 (0.9555) and became champion. After the incident, champion moved back to v1 in one line, with no retraining and no rebuild.

3.2 Monitoring and drift

Four layers Service (p50, p95, p99, errors), data (missing, schema, drift), model (scores, accuracy), business. Percentiles, not means.

SLO and budget Budget = (1 − SLO) × requests. Burn rate = actual / allowed error rate.

Drift types Data (inputs move), label (class shares move), concept (rule changes, invisible to input checks).

PSI and KS PSI = $\Sigma(q - p) \ln(q/p)$: 0.1 watch, 0.25 drift. KS = max CDF gap, very sensitive to sample size.

Decay Concept drift: accuracy 0.95 to 0.57 with PSI 0.01. Data drift: PSI 6.2, accuracy 0.95. Retrain on schedule or trigger through the gate.

Scheduling cron or launchd (macOS, Full Disk Access for Documents), schtasks (Windows), cron or systemd timer (Linux), GitHub Actions schedule anywhere.

Percentile
p99 = the value that 99 percent of requests stay below

Error budget
allowed failures = (1 − SLO) × requests

Population stability index
$PSI = \Sigma_{\text{bins}} (q_i - p_i) \ln(q_i / p_i)$

Kolmogorov Smirnov statistic
$D = \max_x |F_{\text{train}}(x) - F_{\text{live}}(x)|$

TRAP The average latency is 24.5 ms, which sounds fine, but the 99th percentile is 292 ms, so 1 request in 100 is ten times slower than typical. Only 98.27 percent of requests met the 200 ms target, below the 99 percent SLO, while errors used 65 percent of the day's budget.

EXAM A shift of 8 in the mean gives PSI 0.610, clear drift. A wider spread gives 0.246 with no change in the mean, which a check on averages would miss. With 200,000 rows a harmless shift of 0.3 already has a KS p value of 1.3e−12, while PSI is 0.001. Use the size of the change to decide, and the p value only as a hint.

EXAM Under concept drift accuracy fell from 0.95 to 0.57 in a year while the PSI of the input stayed at 0.01. Retraining every 3 months kept it between 0.86 and 0.94. Under data drift the PSI reached 6.20 but accuracy stayed at 0.94 to 0.95, because the model's rule still held.

3.3 Explainability and fairness

Global importance Permutation on test data: I = s − mean shuffled score. Impurity importance is computed on training data and favours many valued features.

Shapley values φ averages what a feature adds over every order. Base + Σφ = prediction. $2^{|F|}$ coalitions, so use TreeExplainer or sampling.

Explanations Show how the model used a feature, not a cause. They depend on the background data. Counterfactuals give actionable changes.

Group metrics Disparate impact = rate B / rate A, flag below 0.8. Equal opportunity: equal TPR. Equalized odds: equal TPR and FPR.

Mitigation Pre processing (data), in processing (constrained loss), post processing (group thresholds). Re-measure everything afterwards.

Impossibility and documentation Different base rates: calibration and equal error rates conflict. Pick a rule for the harm, write a model card, gate promotion on it.

Permutation importance $I_j = s - (1/K) \Sigma_k s_{k,j}$

Shapley value
$\varphi_j = \Sigma_{S \subseteq F \setminus j} [\, |S|! \, (|F| - |S| - 1)! \,/\, |F|! \,] \cdot [\, v(S \cup j) - v(S) \,]$

Efficiency $f(x) = \varphi_0 + \Sigma_j \varphi_j$

Disparate impact ratio
$DI = P(\hat{y} = 1 \mid B) \,/\, P(\hat{y} = 1 \mid A)$

Equal opportunity gap
$TPR_A - TPR_B$, with $TPR = P(\hat{y} = 1 \mid y = 1)$

Chouldechova's identity
$FPR = [p / (1 - p)] \cdot [(1 - PPV) / PPV] \cdot (1 - FNR)$

EXAM Impurity importance always adds up to 1 and gives worst radius 0.442. On test data, shuffling worst radius costs 0.170 accuracy, the real dependence. The random ID column, pure noise, still got 0.018 of the impurity credit because its many distinct values offer many split points on training data. Shuffling it changed the test score by −0.008, which is just noise.

EXAM The base value is 0.450, the average predicted chance of repaying. This applicant's debt ratio of 0.82 pulls the prediction down by 0.194, the low income by 0.121 and the young age by 0.052, ending at 0.083. The three contributions add up exactly from 0.450 to 0.083, which is the efficiency property.

EXAM With one threshold of 0.5 group B is approved 33 percent of the time against 64 percent for group A, a disparate impact of 0.52, far below the four fifths rule of 0.8. Among people who did repay, B was approved 48 percent of the time and A 75 percent. Lowering B's threshold to 0.38 closed the TPR gap, raised the ratio to 0.92 and cost only 0.010 accuracy (0.647 to 0.637).

STEP 4 Design and certify

4.1 Designing an ML system end to end

Design framework Goal and metric, requirements as numbers, data and features, baseline model, serving, offline then online evaluation, monitoring and retraining.

Sizing QPS = daily requests / 86,400, times a peak factor. Little's law: throughput = workers / latency. Replicas = ceil(peak / (capacity × load)).

Serving modes Batch (scheduled, stale), online (ms, tail latency), streaming (fresh features). Retrieve then rank for big catalogues.

Skew Same feature, different code in training and serving. One definition or feature store, logged features, parity test.

Safe release Shadow, canary 1 to 100 percent with guardrails, automatic rollback, keep the old version ready.

A/B tests $n = (z_a + z_b)^2 \, 2\bar{p}(1 - \bar{p}) / \delta^2$. Fix n first, test once. Peeking: 4.4 to 22 percent false alarms.

Average load QPS = daily requests / 86,400

Sample size per group
$n = (z_{1-\alpha/2} + z_{1-\beta})^2 \cdot 2\,\bar{p}(1-\bar{p}) / \delta^2$

Two proportion z test
$z = (p_B - p_A) / \sqrt{(\,p(1-p)(1/n_A + 1/n_B)\,)}$

EXAM Ten million daily users with 20 requests each average 2,315 requests per second and peak at 6,944. One replica with 4 workers at 40 ms handles 100 per second, so 116 replicas keep each at 60 percent load. Ranking a million items per request would take a full second, which is why recommenders retrieve 500 candidates first.

TRAP The model scored 0.730 offline. The serving code filled missing spend with 0 instead of the training median and received spend in pence, not pounds, so live accuracy fell to 0.550 and the mean predicted churn collapsed from 0.435 to 0.124. A parity test on the same raw rows finds the difference on 100 percent of rows before release.

EXAM To detect a lift from 5.0 to 5.5 percent with α = 0.05 and 80 percent power you need 31,235 users per group. In 2,000 A/A tests a single look at the end gave 4.4 percent false alarms, close to the promised 5. Stopping on the first day with $p < 0.05$ gave 22.0 percent, more than four times as many.

EXAM The built-in demo server can work on 4 requests at once, each taking 20 ms, so Little's law predicts 4 / 0.020 = 200 requests per second. The test gets close to 200 with 4 users and stays just under 200 with 8 and 16 users, while the median latency rises from about 21 ms to about 40 ms and then higher. Past the saturation point, more users only add waiting time. Timings change a little on every run.

4.2 MLOps revision and certification practice

The loop Setup, lifecycle, pipelines, serving, Docker and CI/CD, tracking, monitoring, explain and fair, system design, then retrain.

Maturity Level 0 manual, level 1 continuous training through a pipeline, level 2 CI/CD for the pipeline itself.

Release gate Parity, quality gain on the same holdout, fairness (DI ≥ 0.8, TPR gap), score PSI, size. Any FAIL blocks promotion.

Key numbers PSI 0.1 and 0.25. DI 0.8. Burn rate 14.4 for 2 percent in an hour. Peeking 4.4 to 22 percent. Layer cache 2.2 s against 32 s.

Capstone Fingerprint, pipeline, tracked run, alias, tested API (200, 422), drift check, model card.

Cloud mapping Pipelines, endpoints, tracking, registry, monitoring, explainability and feature store exist on Vertex AI, SageMaker and Azure ML under their own names.

PSI $\mathrm{PSI} = \Sigma\,(q - p)\,\ln(q / p)$

Little's law $\text{workers} = \text{request rate} \times \text{latency}$

Disparate impact $\mathrm{DI} = \mathrm{rate}_B / \mathrm{rate}_A \geq 0.8$

Error budget and burn rate
$\text{budget} = (1 - \text{SLO}) \times \text{requests}$, $\text{burn} = \text{actual} / \text{allowed}$

EXAM Challenger 1 has the higher AUC, 0.0498 above the champion, because it uses postcode and the past labels favoured group A. The gate blocks it: the equal opportunity gap is 0.227 and the disparate impact is 0.48. Challenger 2 drops postcode, still gains 0.0125 AUC, passes all six checks and is promoted. A higher score on biased labels is not a better model.

EXAM A common exam pattern: a scenario with stale models and manual retraining, and four services to choose from. The answer is the pipeline plus monitoring service of that platform, because the problem is level 0 maturity, not the model.